

Lab Project

Crack segmentation using Deep Learning

Artificial Vision & Pattern Recognition

Màster en Enginyeria de la Seguretat Informàtica i Intel·ligència Artificial (URV)
Màster interuniversitari en Intel·ligència Artificial (UPC, UB & URV)

Student: Óscar Cubeles Ollé

Abstract

This report presents a study on automatic crack detection and segmentation in structural images using deep learning techniques. The project focuses on the application of a **U-Net-based semantic segmentation model** to identify cracks at the pixel level, aiming to improve the efficiency and reliability of structural health monitoring. The model is developed and trained using the **MATLAB Deep Learning Toolbox** and evaluated on the **DeepCrack dataset**. The performance of the model is assessed on the test set using three evaluation metrics: **accuracy**, **Jaccard index (IoU)**, and **F1-score**.

10th January 2026

1 Introduction

Crack detection in images of various structures, such as bridges, pressure vessels, and machinery, is a critical task for structural health monitoring and safety assessment. Manual inspection is time-consuming, subjective, and prone to errors, especially when cracks are thin or partially occluded. To address this challenge, the DeepCrack method has been proposed.

In this project, a semantic segmentation approach is adopted using the MATLAB Deep Learning Toolbox and a U-Net architecture to perform crack segmentation. The provided DeepCrack dataset is organized into training and testing sets, with separate folders for images and corresponding pixel-level labels (`train_img`, `train_lab`, `test_img`, `test_lab`). The U-Net is trained on the training set and its performance is evaluated on the test set using three commonly used metrics: accuracy, Jaccard index (IoU), and F1-score.

2 Implementation

The crack segmentation framework was implemented using the MATLAB Deep Learning Toolbox, employing a U-Net architecture for semantic segmentation. The dataset, DeepCrack, was organized into training and testing sets, each containing separate folders for images (`train_img`, `test_img`) and corresponding pixel-level labels (`train_lab`, `test_lab`). Each label image contained pixel-wise annotations indicating background or crack regions, which were mapped to numeric masks for training and visualization.

The data preparation involved reading images and labels using `imageDatastore` and `pixelLabelDatastore`, and combining them with `pixelLabelImageDatastore` to facilitate paired access. Images and labels were resized to 256×256 pixels, the input size required by the U-Net, using nearest-neighbor interpolation for labels to preserve categorical information. A preprocessing function was applied to each data pair to perform these resizing operations in a reproducible way.

The U-Net architecture was initialized with two classes, background and crack, and trained using the Adam optimizer for 20 epochs with a mini-batch size of 8. To improve efficiency, the trained network was saved to disk and reused for prediction on the test set when available. Predictions were generated using the `semanticseg` function and resized back to the original image dimensions for evaluation and visualization. Ground truth and predicted masks were converted to numeric formats, with predicted crack regions visualized in yellow over the original images. For quantitative evaluation, the predictions were compared to the ground truth using accuracy, Jaccard index (IoU), and F1-score metrics. Additionally, multiple test images were processed in sequence, with the input image, ground truth, and prediction displayed side by side for qualitative inspection.

3 Results

3.1 Training Results

The training of the U-Net network was monitored through the evolution of accuracy and loss over epochs. Figure 1 shows the complete training progression. During the initial epochs, the network accuracy started from nearly 0%, reflecting the default untrained U-Net performance, and rapidly improved to around 90%. Despite some fluctuations in accuracy during the early iterations, the network converged steadily, reaching a final accuracy of over 99% in the last epochs.



Figure 1: Training progress showing evolution of accuracy and loss over epochs. The initial epochs had the most significant impact, and the network converged by the last epoch.

3.2 Evaluation on Test Set

The trained network was evaluated on the test set using multiple metrics including global accuracy, class accuracy, intersection over union (IoU or Jaccard), weighted IoU, and the BF score. A total of 237 images were processed. Table 1 summarizes the overall dataset metrics.

GlobalAccuracy	MeanAccuracy	MeanIoU	WeightedIoU	MeanBFScore
0.98067	0.82708	0.78801	0.96347	0.84092

Table 1: Dataset-level metrics computed on the test set.

Class-specific metrics are shown in Table 2. The background class shows higher accuracy and IoU values due to the unbalanced nature of the dataset, where background pixels dominate the images. The crack class, being less frequent, shows lower metrics but remains satisfactory.

Class	Accuracy	IoU	MeanBFScore
background	0.9952	0.9801	0.8784
crack	0.6589	0.5959	0.8037

Table 2: Class-level evaluation metrics. Background dominates pixel count, while cracks are less frequent.

Table 2 and Figure 2 visualize the computed metrics. The results indicate strong performance, particularly for the background class.

```

Evaluating semantic segmentation results
-----
* Selected metrics: global accuracy, class accuracy, IoU, weighted IoU, BF score.
* Processed 237 images.
* Finalizing... Done.
* Data set metrics:

```

GlobalAccuracy	MeanAccuracy	MeanIoU	WeightedIoU	MeanBFScore
0.98067	0.82708	0.78801	0.96347	0.84092

Figure 2: Global and Mean metrics on the test set.

Finally, qualitative results for three sample test images are shown in Figures 3–5. Each figure displays the input image, ground truth, and predicted segmentation side by side. Overall, the network outputs

are quite accurate. Some discrepancies, particularly on crack pixels, may result from pixels that could be considered ambiguous. For example, in `testimg1.png`, dark pixels could be considered cracks by the network, while the ground truth does not annotate them as such.

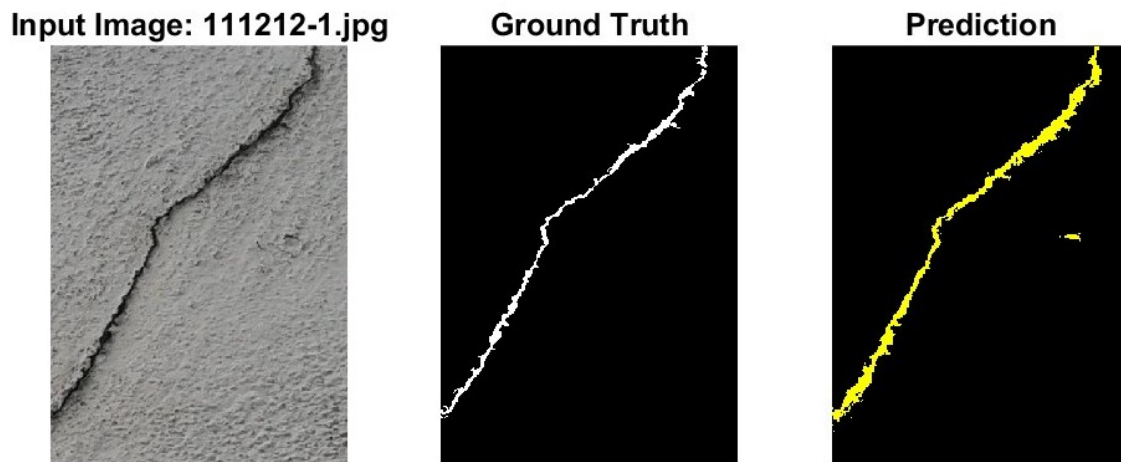


Figure 3: Test image 1. The network detects properly all crack regions even a part that the ground truth does not consider crack but that could be considered crack depending on the conditions.

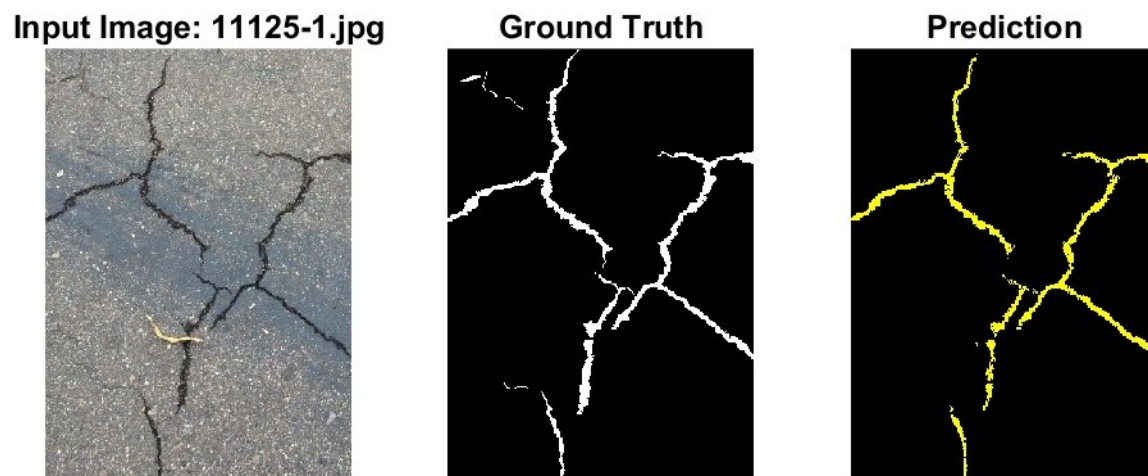
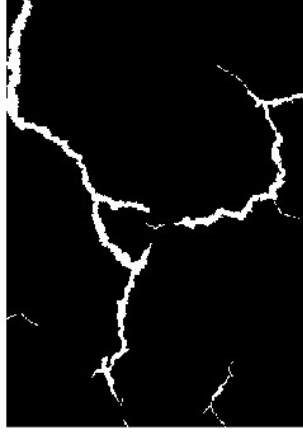


Figure 4: Test image 2. The predicted image misses a small crack at the top left but other regions are satisfactorily detected.

Input Image: 11125-2.jpg



Ground Truth



Prediction

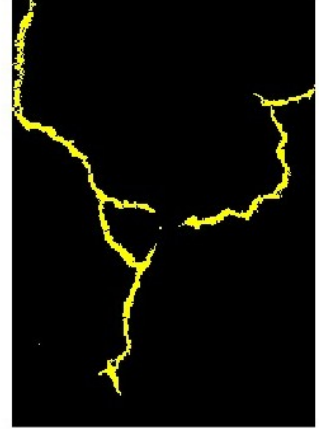


Figure 5: Test image 3. The network detects almost all crack pixels correctly except a couple regions at the bottom of the image which are very light cracks that could even not be considered cracks.

4 Conclusion

In this work, a semantic segmentation approach using a U-Net architecture and the MATLAB Deep Learning Toolbox was applied to the DeepCrack dataset for automatic crack detection. The network was trained on a set of annotated images and evaluated on a separate test set. The training process demonstrated rapid improvement in accuracy during the initial epochs, with the network converging to high performance after 20 epochs. Preprocessing, resizing, and proper handling of label masks were critical steps to ensure effective training and reliable segmentation results.

As a concluding summary, the results show that the U-Net-based segmentation network achieves excellent performance on the dominant background class and satisfactory results for crack detection. The qualitative assessment confirms that most cracks are correctly identified, and minor differences may be attributed to difficult to evaluate pixels in the dataset images. Overall, the approach provides an efficient and reproducible solution for automatic crack segmentation.